Decisions in One Forward Pass: Typed Answers from Local Language Models, and Making Their Yes/No Probabilities Mean Something

Ali Ashter
hello@ashter.dev

October 2026

Abstract

Many uses of language models are decisions, not prose: route a ticket, flag a message, answer yes or no. Asking a model to write its answer as JSON is slow and has to be parsed. We study the alternative of reading typed answers directly from next-token probabilities over single-token answer codes, with all questions about one input batched into one forward pass. On an RTX 5090, six answers about a support ticket take 57 ms with a 4B model, against 1.1 to 2.5 s for the same model writing them as JSON. We then ask what it takes to make these answers good. Across eleven open models on twelve classification tasks, untrained models are badly overconfident and four training-free prompt changes do not close the gap. For yes/no questions, which recent work has shown to be poorly calibrated, we test the corrections that work proposed but did not evaluate. Averaging a question with its complement, which makes the probabilities coherent by construction, barely improves calibration and hurts ranking when the opposite is phrased as a negation. Plain Platt scaling does what is needed: across four open models and six tasks, 25 labels cut mean calibration error from 0.197 to 0.089 and raise accuracy from 0.779 to 0.845; 100 labels bring every model-task pair below 0.075. Finally, a rank-16 LoRA trained for 74 minutes on 15 datasets disjoint from the benchmark raises a 4B model's mean score from 0.713 to 0.756 and cuts its calibration error from 0.241 to 0.108. Code, data builders and every score are public at `https://github.com/alleyxd/AnyDecide`.

1 Introduction

A support system that reads a message and must decide which team handles it, how urgent it is and whether the customer is about to leave does not need a paragraph. It needs three typed values, ideally with probabilities it can threshold. The common way to get them from a language model is to ask for JSON and parse it. That costs one decoding step per output token, can fail to parse, and gives no usable confidence.

A decision can instead be read in a single step. If each option has a single-token code (A, B, C; Yes or No; 0 to 9), the model's next-token distribution restricted to those codes is a probability over the options, always of the requested type. Questions about the same input differ only in their last lines, so they can share one batched forward pass. Commercial "decision models" are built around this idea [1], and open projects have followed [2]. Two practical questions remain open for anyone who wants to do this with a model they run themselves: how good are those probabilities out of the box, and what is the cheapest way to fix them?

Our contributions:

- **A minimal implementation and speed measurements.** Six typed answers about one input cost about as much as one (57 ms vs 46 ms on a 4B model), 20 to 45 times faster than generating the same answers as JSON.
- **A baseline across eleven open models** on twelve public classification and routing tasks, scored with identical prompts and samples, and four training-free changes that do not help: scoring option names instead of codes, few-shot examples, permuting options, and two-stage choice for large label sets.
- **A test of corrections for yes/no calibration.** Recent work reports that yes/no probabilities from decision models and open LLMs are miscalibrated and incoherent ($P(X) + P(\neg X) \neq 1$) but evaluates no fix [3, 4]. We compare complement averaging, temperature scaling, Platt scaling, a two-feature variant and isotonic regression across four models and six binary tasks, as a function of the number of labelled examples.
- **Training on disjoint data.** A small LoRA trained on 36k decision questions from 15 datasets that do not appear in the benchmark improves accuracy where untrained models are most biased and halves calibration error, and we report where it hurts.

2 Related work

Decision models. Jev [1] is a closed model that answers typed questions (choice, score and probability, in its terms) from one pass; Deusser et al. evaluate it on 37 datasets and find high AUROC but poorly calibrated yes/no probabilities, with per-question threshold tuning as the only fix tested [3]. Kev [2] is an open family of LoRA-tuned Qwen models with a pointer head and a fitted temperature, trained extensively and reported within a few points of Jev. Our goal is narrower: a small, dependency-light way to do this with any Hugging Face model, and a careful look at calibration fixes.

Calibration. Temperature scaling [5] and Platt scaling [6] fit one or two parameters on held-out labels; isotonic regression [7] fits a monotone map. Contextual calibration [8] corrects a model's prior over labels without labels. Language models' token probabilities can be informative about correctness [9], but chat tuning makes them overconfident. Li et al. show that yes/no probabilities violate the complement axiom by up to 0.29 for open models [4].

Constrained output. Grammar-constrained decoding guarantees valid structure [10] but still generates token by token; reading a single next-token distribution avoids decoding altogether.

3 Method

A question has a type: *choice* (one of k options), *score* (one of up to ten ordered levels) or *yes/no*. It is rendered into a fixed prompt: the input ("state"), the question, the options with their codes (A, B, …; levels 0 to 9; or Yes/No), and an instruction to reply with the code only. The prompt goes through the model's chat template with thinking disabled. We take the logits of the next token, keep the entries of the answer codes, and apply a softmax. All questions about one state are padded into one batch and run in a single forward pass that keeps only the last position's logits. Every code must be a single token in the model's tokenizer, which holds for the common open families; the library checks this and fails loudly otherwise. The output is always of the requested type, with a probability per option, a score's expected level, or $P(\text{yes})$.

4 Speed

Table 1 shows the effect of batching: six answers cost 1.2 times one. Generated JSON grows with the number of fields and with the model's verbosity; one-pass decisions do not.

	Qwen3-4B	tuned 4B
1 question	46 ms	50 ms
6 questions, one batch	57 ms	57 ms
6 answers written as JSON	2,543 ms	1,138 ms
Speed-up	45×	20×

Table 1: Latency for six questions about one short support ticket, RTX 5090, bf16, Hugging Face Transformers. The tuned model writes shorter JSON, which is why its baseline is faster.

Model (untrained)	Mean of 12
Qwen3.5-9B	0.781
Qwen3.5-4B	0.747
Qwen2.5-7B-Instruct	0.746
Qwen3-4B	0.713
Mistral-7B-Instruct-v0.3	0.667
Llama-3.1-8B-Instruct	0.652
Phi-4-mini	0.611
Qwen3.5-2B	0.571
Qwen3-1.7B	0.535
Qwen3-0.6B	0.433

Table 2: Mean accuracy (F1 for spam and toxic chat) over twelve tasks, 300 examples each, one-pass readout.

5 How good are untrained models?

We use twelve public tasks: SST-2, Rotten Tomatoes, emotion, financial phrasebank, AG News, Banking77 (77 intents), CLINC150 (150 intents), SMS spam and toxic chat (both scored by F1), BoolQ, ANLI and PAWS, with the same 300 examples and prompts for every model, through an existing benchmarking harness for decision models.

Size helps but is not everything (Table 2): Qwen3.5-4B matches Qwen2.5-7B and beats both 7 to 8B models from other families. The main failure is calibration. Untrained chat models put near-certain probability on almost every answer; the speed test above shows 1.0 on every one of six questions, including uncertain ones.

Training-free changes that did not help. On Qwen3.5-4B we tried scoring the option names instead of letter codes (worse on four of five tasks, e.g. CLINC150 0.680 vs 0.723, and three to ten times slower), two to eight few-shot examples (mean 0.748 zero-shot, 0.762 with four, 0.772 with eight, with gains concentrated on financial phrasebank and losses on three tasks), averaging over three option orderings (0.738 vs 0.722 on the same eight tasks) and a two-stage choice for the large intent sets (no gain). A label-free prior correction [8] was unreliable and hurt the imbalanced tasks (toxic chat F1 0.765 to 0.571). Fitting a per-option bias on 150 labels helped consistently, by 2 to 12 points (financial phrase-

Method	ECE	Brier	AUROC	Acc
Raw $\sigma(z)$	0.197	0.198	0.889	0.779
Complement avg., opposite	0.178	0.183	0.886	0.787
Complement avg., negation	0.165	0.186	0.837	0.789
Temperature, n=100	0.141	0.145	0.907	0.779
Platt, n=25	0.089	0.119	0.903	0.845
Platt, n=50	0.070	0.110	0.906	0.851
Platt, n=100	0.060	0.107	0.907	0.854
Platt, n=200	**0.055**	**0.106**	0.907	**0.855**
Platt on (z, z_{opp}), n=100	0.059	0.107	0.906	0.854
Isotonic, n=25	0.096	0.122	0.885	0.840
Isotonic, n=200	**0.055**	0.107	0.905	0.854

Table 3: Yes/no calibration, mean over four models and six tasks (24 pairs), 500 test examples per pair, fitted methods averaged over ten random splits.

bank 0.543 to 0.800), which motivates the calibration study.

6 Making yes/no probabilities mean something

Setup. Six binary tasks: SST-2, Rotten Tomatoes, BoolQ, SMS spam, PAWS and MRPC, 1,000 examples each (all 872 for SST-2). Every example is asked three ways: the question ("Is this review positive?"), a natural opposite ("Is this review negative?") and a plain negation ("Is it false that this review is positive?"). We record the margin $z = \log P(\text{Yes}) - \log P(\text{No})$ for each and compare: the raw probability $\sigma(z)$; complement averaging $\frac{1}{2}(\sigma(z) + 1 - \sigma(z'))$ with either opposite, which is coherent by construction; temperature scaling $\sigma(z/T)$; Platt scaling $\sigma(az+b)$; a two-feature Platt model on (z, z_{opp}); and isotonic regression. Fitted methods use $n \in \{25, 50, 100, 200, 400\}$ labels and are tested on 500 held-out examples, averaged over ten random splits. We report expected calibration error (15 bins), Brier score, AUROC and accuracy at 0.5. Models: Qwen3-4B, Qwen3-8B, Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.3, all used as-is.

Results. Table 3 averages the 24 model-task pairs; per-pair numbers are in the repository. Four findings hold across models.

Raw probabilities are badly calibrated, and the worst cases are biases, not overconfidence. Raw ECE ranges from 0.068 to 0.604. The largest errors are on SMS spam (0.59 for Llama-3.1-8B, 0.60 for Qwen3-4B), where the models call far too many messages spam.

Complement averaging is not a fix. The answers really are incoherent: $\sigma(z) + \sigma(z')$ misses 1 by 0.18 to 0.30 on average with the natural opposite, and by up to 0.77 (Mistral) with a plain negation. But averaging with the opposite cuts mean ECE only from 0.197 to 0.178 and makes it worse on 7 of the 24 pairs (Llama-3.1-8B on MRPC: 0.130 to 0.268). With the negated question it also damages ranking: AUROC falls from 0.889 to 0.837, because the model's answer to an awkwardly negated question is a noisier signal than its answer to the original. Coherence by construction does not imply calibration.

Temperature alone is not enough. A single temperature can only sharpen or soften; it cannot move the decision point. It leaves the biased spam cases at ECE 0.39 to 0.47 and does not change accuracy.

Two parameters and a few dozen labels are. Platt scaling, which adds a bias to the temperature, halves mean ECE with 25 labels, reaches 0.060 with 100, and with 100 labels keeps every one of the 24 pairs below 0.075. Because it moves the threshold, it also raises accuracy at 0.5, by 7.5 points on average with 100 labels (3.5 for Mistral to 10.5 for Qwen3-4B), while leaving AUROC unchanged: the model already ranks examples well, its probabilities are simply offset. Isotonic regression needs more labels to match it, and using the opposite question as a second feature adds nothing.

Training helps, and Platt still helps after it. We ran the same study on Qwen3-4B with the decision LoRA of the next section, trained on none of these six datasets. Raw mean ECE falls from 0.247 to 0.107, accuracy from 0.749 to 0.838 and AUROC from 0.863 to 0.926, and five of the six tasks are already at ECE 0.046 to 0.080 without any fitting. The exception is PAWS (0.344), the same task where training hurt accuracy; 100 labels of Platt scaling bring it to 0.072 and the six-task mean to 0.055. The tuned model is markedly worse with negated questions, though: complement averaging with the negation raises its ECE to 0.260, so training on positively phrased questions narrows what the model handles well.

7 Training a decision model on disjoint data

We build 36,194 decision questions from 15 public datasets that do not appear in the benchmark (DBpedia, Yahoo Answers, Amazon and Yelp reviews, SNLI, MNLI, QQP, QNLI, four TweetEval tasks, MASSIVE intents, Civil Comments, financial tweets and 20 Newsgroups), each rendered with exactly the inference prompt, with the answer code as the target. One epoch of LoRA (rank 16) on Qwen3-4B takes 74 minutes on an RTX 5090 at 10.6 GB peak memory.

Training helps most where the untrained model was most biased: routing among 150 intents (+20 points) and spam (+22 points F1), and it more than halves calibration error (Table 4). It hurts on PAWS (−9.4) and ANLI (−4.0). PAWS pairs share almost all their words, and the training data's paraphrase source (QQP) does not, so the tuned model learns that word overlap means paraphrase.

Task	Qwen3-4B	+ LoRA
CLINC150 (150 intents)	0.593	**0.793**
SMS spam (F1)	0.553	**0.776**
Financial phrasebank	0.737	**0.837**
Banking77	0.610	**0.683**
BoolQ	0.853	**0.880**
SST-2	0.947	**0.963**
Rotten Tomatoes	0.883	**0.893**
Emotion	0.540	**0.553**
AG News	0.877	0.877
Toxic chat (F1)	**0.750**	0.737
ANLI	**0.543**	0.503
PAWS	**0.667**	0.573
Mean	0.713	**0.756**
Mean ECE	0.241	**0.108**

Table 4: Effect of the decision LoRA, same 300 examples per task. None of these datasets is in the training data.

8 Limitations

Accuracy numbers use 300 examples per task, so differences under about three points are within noise. All timings come from one consumer GPU under Windows with Hugging Face Transformers; a serving engine would be faster for both methods. We trained one model size. Knowledge-heavy tasks are out of scope: one-pass readout cannot make a small model know more. Prompts were not tuned per task, by design.

9 Conclusion

Reading decisions from one forward pass is fast enough to put a language model in places where generation is too slow, and the answers always have the right type. Out of the box the probabilities are not trustworthy. For yes/no questions, making the answer coherent does not make it calibrated; fitting two parameters on 25 to 100 labelled examples does, and it is the cheapest improvement available. A short LoRA on unrelated public data makes a 4B model a better and much better-calibrated decision maker, except on tasks that reward ignoring word overlap.

References

[1] TypeSafe. Jev, a System One decision model. Product release, September 2026.

[2] J. Palmer. Kev: open decision models. `https://github.com/jaredpalmer/kev`, 2026.

[3] T. Deusser, L. Sparrenberg, R. Sifa. Evaluating and benchmarking the System One model Jev. arXiv:2609.37647, 2026.

[4] Li, He, Li. Beyond calibration: do a typed-decision model's probabilities obey the probability axioms? arXiv:2609.33209, 2026.

[5] C. Guo, G. Pleiss, Y. Sun, K. Weinberger. On calibration of modern neural networks. ICML, 2017.

[6] J. Platt. Probabilistic outputs for support vector machines. Advances in Large Margin Classifiers, 1999.

[7] B. Zadrozny, C. Elkan. Transforming classifier scores into accurate multiclass probability estimates. KDD, 2002.

[8] Z. Zhao, E. Wallace, S. Feng, D. Klein, S. Singh. Calibrate before use: improving few-shot performance of language models. ICML, 2021.

[9] S. Kadavath et al. Language models (mostly) know what they know. arXiv:2207.05221, 2022.

[10] B. Willard, R. Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.